Subgraph Representation Learning: The Main Methods

October 13, 2026

The problem

There is one large graph. A *subgraph* is a set of its nodes, for example the visits of one patient. Each subgraph has a label, and the task is to predict it. Methods for this task are called *subgraph representation learning*: they turn each subgraph into one vector, and a classifier reads the label from that vector.

The task sits between two better-known ones. In *node classification* the thing to label is a single node. In *graph classification* it is a whole small graph, such as a molecule, that stands alone. Here the thing to label is a group of nodes that lives inside one big graph, so the nodes and edges around it can matter as much as the nodes in it.

Why a plain GNN is not enough. The simple approach is to run a graph neural network (GNN) on the whole graph, which gives every node a vector, and then average the vectors of the subgraph's nodes. The weakness is that the GNN does not know which nodes belong to the subgraph. A node's vector is the same whichever subgraph we are asking about, so the model cannot tell a neighbour inside the subgraph from one outside it. Most methods below are ways around this, or ways to do it more cheaply.

The methods at a glance

Method	Where, year	Main idea
SubGNN	NeurIPS 2020	Messages to each subgraph from sampled small patches of the graph, in three channels: position, neighbourhood and structure.
GLASS	ICLR 2022	Mark every node as inside or outside the subgraph, then run a plain GNN that treats the two kinds differently.
S2N	ICML 2024	Turn every subgraph into one node of a new small graph, then run a simple GNN on it.
GPEN	ICML 2025	Give each node a position in a tree over the whole graph, so far-away structure is used.

Table 1: The main methods, oldest first.

Each method

SubGNN (Alsentzer et al.) The first method, and the source of the standard datasets. It compares each subgraph with small random pieces of the graph, and asks three questions: where the subgraph sits, what is around it, and what shape it has. It is slow, because the pieces must be prepared in advance.

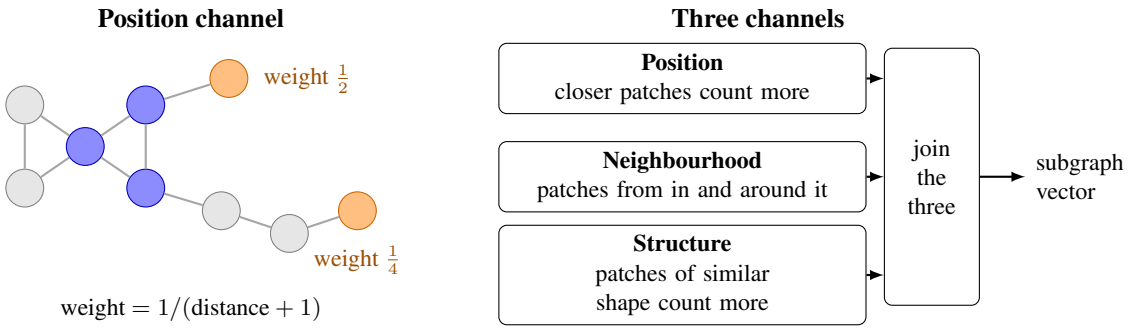


Figure 1: How SubGNN works. Left: orange patches send messages to the blue subgraph. A patch one step away gets weight $\frac{1}{2}$, a patch three steps away gets $\frac{1}{4}$. Right: the same idea in three channels, each with its own patches and its own weight. Each channel is run twice, for the inside of the subgraph and for its border. Patches and weights are computed before training.

GLASS (Wang and Zhang) It tells the GNN which nodes are in the subgraph: each node gets a mark, 1 inside and 0 outside. The GNN treats the two kinds differently, so the same node can look different for different subgraphs. Simpler and faster than SubGNN, and better on all datasets.

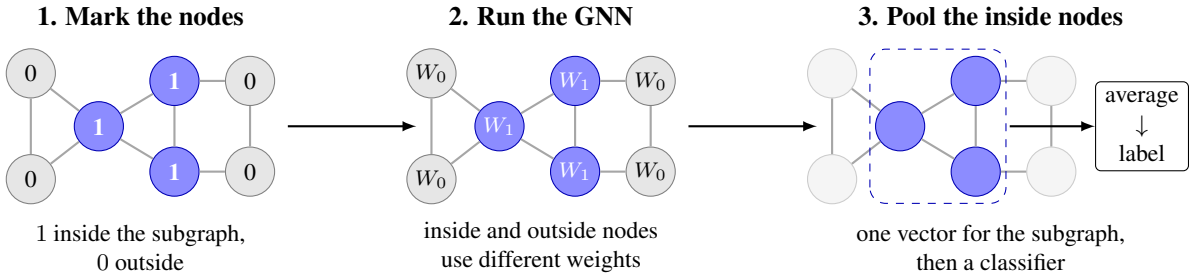


Figure 2: How GLASS works. Blue nodes are the subgraph. The marks change for every subgraph, so the GNN runs again for each one (or for each batch of subgraphs).

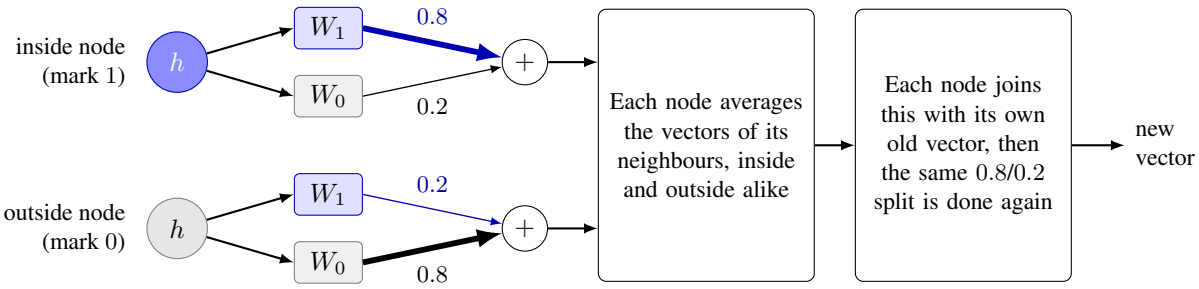


Figure 3: One GLASS layer, as written in the authors' code. Every node goes through both weight sets, W_1 and W_0. The mark only decides the mix: inside nodes take mostly W_1, outside nodes mostly W_0. The 0.8 is a fixed setting, not learned. The mark is not given to the network as a feature; this split is the only place it is used.

S2N (Kim and Oh) It turns each subgraph into one node of a new, small graph, and joins two such nodes when their subgraphs are linked. A plain GNN then labels the nodes of this small graph. As good as GLASS,

and 183 to 711 times faster.

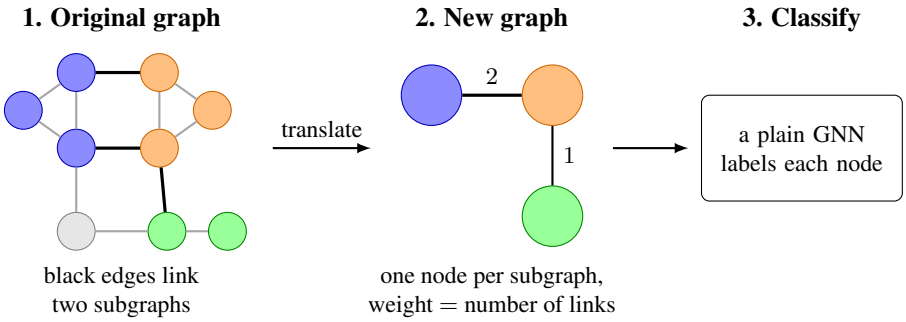


Figure 4: How S2N works. Each subgraph becomes one node. Two new nodes are joined when edges link their subgraphs, and the edge weight is the number of such edges. Blue and green are linked only through a gray node, so they get no edge. Each new node starts from a weighted sum of its nodes' features, or from a small GNN run on the subgraph alone.

GPEN (Wu et al.) It adds information from far away in the graph. It builds one tree over the whole graph and gives each node a position: its distance from the root. The newest method, and the best on all four real datasets.

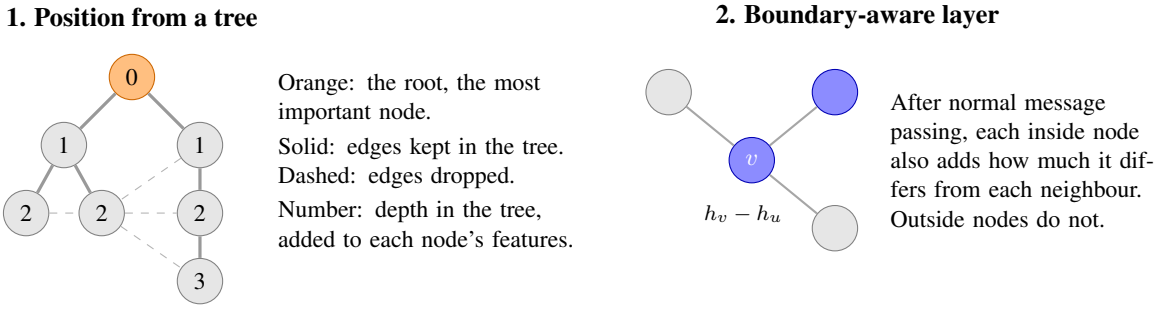


Figure 5: How GPEN works. Left: nodes are scored by importance (similar to PageRank), the top node becomes the root, and a tree is built from the strongest edges. Each node's depth is added to its features. Right: for an inside node v, the layer output is $(1-\beta)\,h_v + \beta \sum_u (h_v - h_u)$, summed over its neighbours u. For an outside node it is $(1-\beta)\,h_v$. β is a fixed setting.

Published results

	PPI-BP	HPO-Metab	HPO-Neuro	EM-User
SubGNN	59.9 ± 2.5	53.7 ± 2.5	64.4 ± 1.9	81.6 ± 4.1
GLASS	61.9 ± 2.2	61.4 ± 1.6	68.5 ± 1.6	88.8 ± 1.9
S2N	64.3 ± 4.1	$\mathbf{63.9 \pm 7.2}$	68.6 ± 1.9	89.0 ± 3.5
GPEN	$\mathbf{64.4 \pm 0.9}$	$\mathbf{63.9 \pm 0.9}$	$\mathbf{69.1 \pm 0.6}$	$\mathbf{91.2 \pm 1.3}$

Table 2: Real datasets. Micro-F1 on the test set, mean and spread over 10 runs, from Table 1 of the GPEN paper. For the binary EM-User this is the accuracy.

	Density	Cut ratio	Coreness	Component
SubGNN	91.9 ± 1.9	62.9 ± 4.1	65.9 ± 9.8	95.8 ± 10.1
GLASS	93.0 ± 2.8	93.5 ± 1.9	84.0 ± 2.8	$\mathbf{100.0 \pm 0.0}$
S2N	$\mathbf{96.3 \pm 5.7}$	89.2 ± 4.4	72.6 ± 11.4	$\mathbf{100.0 \pm 0.0}$
GPEN	95.6 ± 1.6	$\mathbf{93.6 \pm 1.6}$	$\mathbf{87.6 \pm 1.9}$	$\mathbf{100.0 \pm 0.0}$

Table 3: Synthetic datasets. Micro-F1 on the test set, mean and spread over 10 runs, from Table 3 of the GPEN paper.

What the methods have in common

Everyone uses the same datasets. SubGNN, GLASS, S2N and GPEN all report on SubGNN's four real datasets and its four synthetic ones. Only one of the real ones, EM-User, has a binary label.

Knowing what is inside matters. GLASS showed that just marking the subgraph's nodes lifts a plain GNN by a large margin, to more than double the score on one dataset. Most later methods either keep this idea or find another way to give the model the same information.

Cost is the second big topic. SubGNN needs heavy precomputation, and GLASS must run the GNN again for each small batch of subgraphs, because the marks change with the batch. S2N was made mainly to be cheaper.

Gains have become small. On the real datasets, from GLASS to GPEN the best scores moved by at most 2.5 points. GPEN beats S2N by at most 2.2 points, which is less than the spread of S2N's runs. On the synthetic datasets, Component is solved by all three newer methods, and the only clear gap left is on Coreness, where GPEN leads GLASS by 3.6 points.